

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

**«Белорусский государственный университет информатики и
радиоэлектроники»**

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

ОТЧЕТ

по лабораторной работе №1

по дисциплине

«Технологии компонентного программирования»

«Сериализация и десериализация объектов»

Выполнил:

Лебедев Артем Владимирович
магистрант группы 556301

Преподаватель:

Герман Олег Витольдович
кандидат технических наук
доцент кафедры ИТАС

Минск, 2026 г.

СОДЕРЖАНИЕ

1	Цель работы	2
2	Теоретические сведения.....	3
2.1	Сериализация и десериализация.....	3
2.2	Формат pickle.....	3
2.3	Формат JSON.....	3
2.4	Передача по сети.....	4
2.5	Сериализация визуальных объектов.....	4
3	Реализация	5
3.1	Структура программы.....	5
3.2	Класс «Студент»	5
3.3	Сериализация в pickle и JSON	5
3.4	Десериализация в другом приложении	6
3.5	Клиент-серверное приложение	6
3.6	Сериализация визуальной формы	7
3.7	Графическое приложение	7
4	Дополнительные задания	8
4.1	Удалённый вызов модуля	8
4.2	Компоненты wxPython	8
4.3	Рефлексия.....	8
5	Результаты работы программы	10
6	Ответы на контрольные вопросы	12
	Заключение	13
	Список использованных источников.....	14

1 ЦЕЛЬ РАБОТЫ

Изучить механизмы сериализации и десериализации объектов в языке Python. Создать класс «Студент», сохранить его объект в файл, восстановить в другом приложении, передать по сети в клиент-серверном приложении, сериализовать состояние визуальной формы. Выполнить все пункты отдельно для форматов pickle и JSON. Дополнительно передать вместе с объектом изображение в формате WebP.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Сериализация и десериализация

Сериализация — преобразование объекта, существующего в памяти программы, в последовательность байтов, которую можно записать в файл или передать по сети. Десериализация — обратное преобразование: по последовательности байтов строится объект с теми же данными. Восстановленный объект является новым объектом, а не тем же самым: совпадают значения полей, но не адрес в памяти.

Сериализация нужна, чтобы объект пережил границу процесса: для сохранения состояния между запусками, обмена данными между программами и машинами, кэширования и постановки сообщений в очереди. Она же лежит в основе компонентных технологий: при удалённом вызове аргументы и результат метода передаются именно в сериализованном виде.

2.2 Формат pickle

Модуль `pickle` входит в стандартную библиотеку Python и сохраняет почти любые объекты, включая экземпляры пользовательских классов. Формат двоичный, поэтому файлы открываются в режимах `wb` и `rb`. Функции `dump` и `dumps` преобразуют объект в байты, `load` и `loads` — обратно.

В поток записываются данные экземпляра и имя класса в виде «модуль.класс», но не код класса. Поэтому при восстановлении класс должен быть доступен для импорта, а методы объекта берутся из этого класса. Формат понятен только Python и небезопасен: десериализация данных из недоверенного источника может привести к выполнению произвольного кода.

2.3 Формат JSON

JSON — текстовый формат, который читается человеком и поддерживается всеми языками программирования. Он знает только словари, списки, строки, числа, логические значения и `null`, поэтому объект пользовательского класса нужно явно преобразовать в словарь при записи и собрать обратно при чтении. Методы в JSON не попадают: они появляются, когда из словаря создаётся объект класса.

Двоичные данные в JSON напрямую не записываются. Изображение кодируется в `base64` — текстовое представление, в котором каждые три байта заменяются четырьмя символами, поэтому объём растёт примерно на треть.

Сравнение форматов приведено в таблице 2.1.

Таблица 2.1 – Сравнение форматов pickle и JSON

Свойство	pickle	JSON
Вид данных	двоичный	текстовый
Пользовательские классы	сохраняются автоматически	нужно преобразование в словарь
Изображение	исходные байты	строка base64
Другие языки	только Python	любой язык
Безопасность	нельзя читать чужие данные	безопасен

2.4 Передача по сети

Протокол TCP передаёт поток байтов без границ сообщений: один вызов `recv` может вернуть только часть отправленного. В примере из методических материалов используется `recv(1024)`, чего достаточно для небольшого словаря, но не для объекта с изображением. Поэтому в работе перед данными передаётся их длина, и получатель читает ровно столько байтов, сколько объявлено. Формат сообщения показан в таблице 2.2.

Таблица 2.2 – Формат сообщения

Поле	Размер	Содержимое
Длина	4 байта	длина остальной части, целое без знака, сетевой порядок байтов
Формат	8 байт	строка <code>pickle</code> или <code>json</code> , дополненная нулями
Данные	переменный	сериализованный объект

2.5 Сериализация визуальных объектов

Элементы окна (кнопки, поля) связаны с ресурсами операционной системы и напрямую не сериализуются. Сохраняется снимок состояния: заголовок окна, надпись на кнопке, текст в поле ввода. По такому снимку окно создаётся заново — этот же приём показан в методических материалах на примере методов `to_dict` и `from_dict`.

3 РЕАЛИЗАЦИЯ

3.1 Структура программы

Листинг 3.1 – Структура лабораторной работы

lab1/models.py	классы Student, Photo и GuiFormState
lab1/codecs.py	сериализация в pickle и JSON, изображение в base64
lab1/net.py	сообщения с указанием длины
lab1/server.py	TCP-сервер: принимает объект и возвращает его
lab1/client.py	TCP-клиент: отправляет объект и читает ответ
lab1/reader.py	отдельное приложение, читающее файл с объектом
lab1/cases.py	проверяемые сценарии
lab1/gui.py	окно: карточка студента и список сценариев
lab1/gui_form.py	окно, восстановленное из снимка состояния
lab1/runner.py	запуск сценариев из командной строки
lab1/extras/	дополнительные задания

3.2 Класс «Студент»

Класс Student (листинг 3.2) содержит три поля из задания и необязательную фотографию. Метод greet нужен, чтобы показать, что после восстановления объект сохраняет поведение.

Листинг 3.2 – Классы предметной области

```
@dataclass
class Photo:
    filename: str
    mime: str
    data: bytes

@dataclass
class Student:
    name: str
    group: str
    faculty: str
    photo: Photo | None = None

    def greet(self) -> str:
        return f"{self.name}, {self.group}, {self.faculty}"
```

3.3 Сериализация в pickle и JSON

Для pickle достаточно вызова pickle.dumps: объект вместе с вложенной фотографией сохраняется целиком, изображение — как исходные байты. Для JSON написано преобразование в словарь (листинг 3.3), где изображение кодируется в base64, а рядом записываются имя файла и тип содержимого.

Листинг 3.3 – Преобразование объекта для JSON

```
def student_to_jsonable(student: Student) -> dict[str, Any]:
    payload: dict[str, Any] = {
```

```

        "name": student.name,
        "group": student.group,
        "faculty": student.faculty,
        "photo": None,
    }
    if student.photo is not None:
        payload["photo"] = {
            "filename": student.photo.filename,
            "mime": student.photo.mime,
            "encoding": "base64",
            "data": base64.b64encode(student.photo.data).decode("ascii"),
        }
    return payload

```

3.4 Десериализация в другом приложении

Модуль `lab1/reader.py` запускается отдельным процессом командой `python -m lab1.reader artifacts/student.json`. Он определяет формат по расширению файла, восстанавливает объект и вызывает его метод. Для файла `pickle` при этом проверяется, что получен именно объект класса `Student`.

3.5 Клиент-серверное приложение

Функции приёма и передачи сообщений приведены в листинге 3.4. Функция `recv_exact` читает из сокета в цикле, пока не получит заданное число байтов, и тем самым устраняет ограничение в 1024 байта. Сервер восстанавливает объект, передаёт его в окно программы и возвращает клиенту то же сообщение; клиент восстанавливает объект из ответа.

Листинг 3.4 – Сообщения с указанием длины

```

HEADER = struct.Struct("!I")

def send_message(sock: socket.socket, payload: bytes) -> None:
    if len(payload) > MAX_PAYLOAD:
        raise ValueError(f"payload too large: {len(payload)} bytes")
    sock.sendall(HEADER.pack(len(payload)) + payload)

def recv_exact(sock: socket.socket, size: int) -> bytes:
    chunks = bytearray()
    while len(chunks) < size:
        piece = sock.recv(size - len(chunks))
        if not piece:
            raise ConnectionError("socket closed while reading")
        chunks.extend(piece)
    return bytes(chunks)

def recv_message(sock: socket.socket) -> bytes:
    (length,) = HEADER.unpack(recv_exact(sock, HEADER.size))
    if length > MAX_PAYLOAD:
        raise ValueError(f"declared payload too large: {length}")

```

```
return recv_exact(sock, length)
```

3.6 Сериализация визуальной формы

Состояние формы с кнопкой и текстовым полем описывается классом `GuiFormState` (листинг 3.5). Снимок записывается в файл `artifacts/gui_form.json`, после чего по нему открывается новое окно.

Листинг 3.5 – Снимок состояния визуальной формы

```
@dataclass
class GuiFormState:
    title: str = "Visual form"
    button_label: str = "Click me"
    text_value: str = ""
    label_text: str = "Welcome"
    extra: dict[str, str] = field(default_factory=dict)

    def to_dict(self) -> dict[str, object]:
        return {
            "title": self.title,
            "button_label": self.button_label,
            "text_value": self.text_value,
            "label_text": self.label_text,
            "extra": self.extra,
        }
```

3.7 Графическое приложение

Окно написано на библиотеке `wxPython`. Слева находится карточка студента с полями и фотографией, справа — список сценариев, каждый из которых соответствует пункту задания и запускается кнопкой `Run` либо все сразу кнопкой `All`. Изображение WebP для предварительного просмотра декодируется библиотекой `Pillow`.

4 ДОПОЛНИТЕЛЬНЫЕ ЗАДАНИЯ

Три дополнительные темы выполнены как продолжение работы и размещены в каталоге `lab1/extras`.

4.1 Удалённый вызов модуля

В каталоге `lab1/extras/rpc` реализован сервер XML-RPC на основе стандартного класса `SimpleXMLRPCServer` с методами `add`, `mul`, `greet_student` и `inspect_module`. Клиентов два: на Python (`xmlrpc.client`) и на Java без сторонних библиотек. Связь с основной работой прямая: при удалённом вызове аргументы и результат сериализуются в XML автоматически. Окно показано на рисунке 4.1.

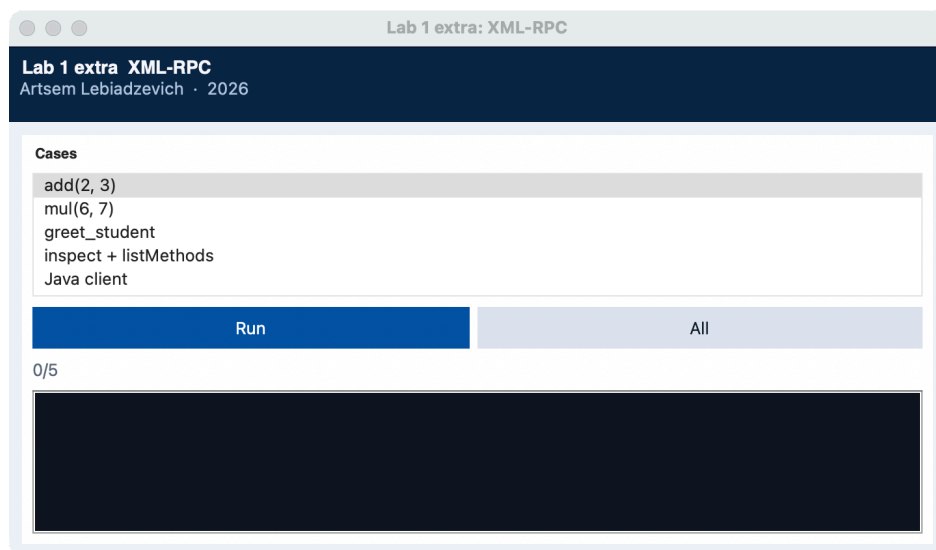


Рисунок 4.1 – Дополнительное задание: удалённый вызов через XML-RPC

4.2 Компоненты wxPython

В каталоге `lab1/extras/wxui` собрано окно с панелью инструментов: просмотр изображения, проигрывание звука, встроенный браузер, календарь с часами, чтение PDF, запуск Word и Excel, а также собственный компонент `CustomButton` — наследник стандартной кнопки. Общие элементы оформления из этого задания используются всеми окнами лабораторных работ.

4.3 Рефлексия

В каталоге `lab1/extras/reflection` класс `Student` создаётся во время выполнения функцией `type`, его методы и сигнатуры исследуются модулем `inspect`, новые методы добавляются объекту через `types.MethodType`. Программа на Java передаёт имя метода строкой, а Python находит и вызывает его функцией `getattr`. Окно показано на рисунке 4.2.

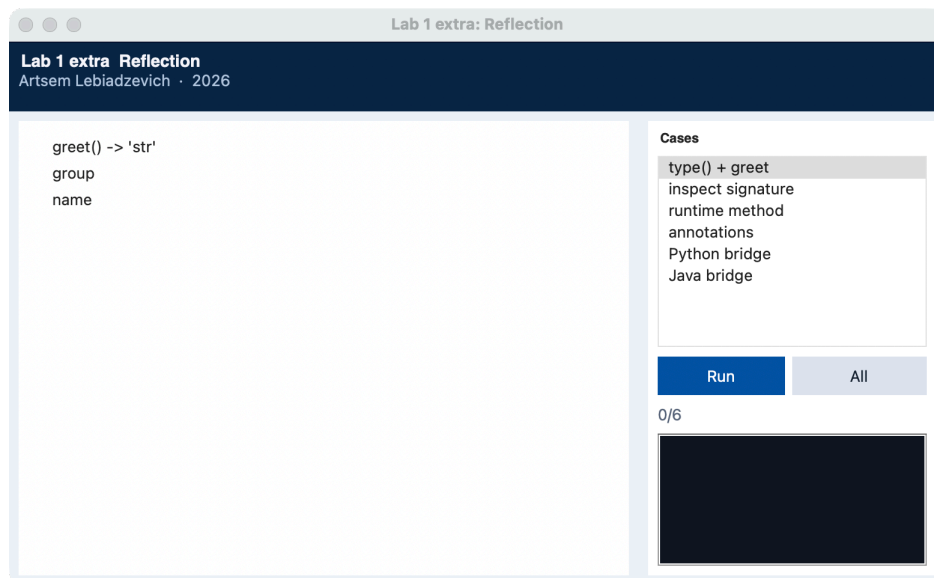


Рисунок 4.2 – Дополнительное задание: рефлексия

5 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

Окно запускается командой `python main.py --lab 1 --gui` (рисунок 5.1). В журнале справа видно, что объект с фотографией размером 6122 байта записан в оба формата, прочитан другим процессом и передан по сети без потерь.

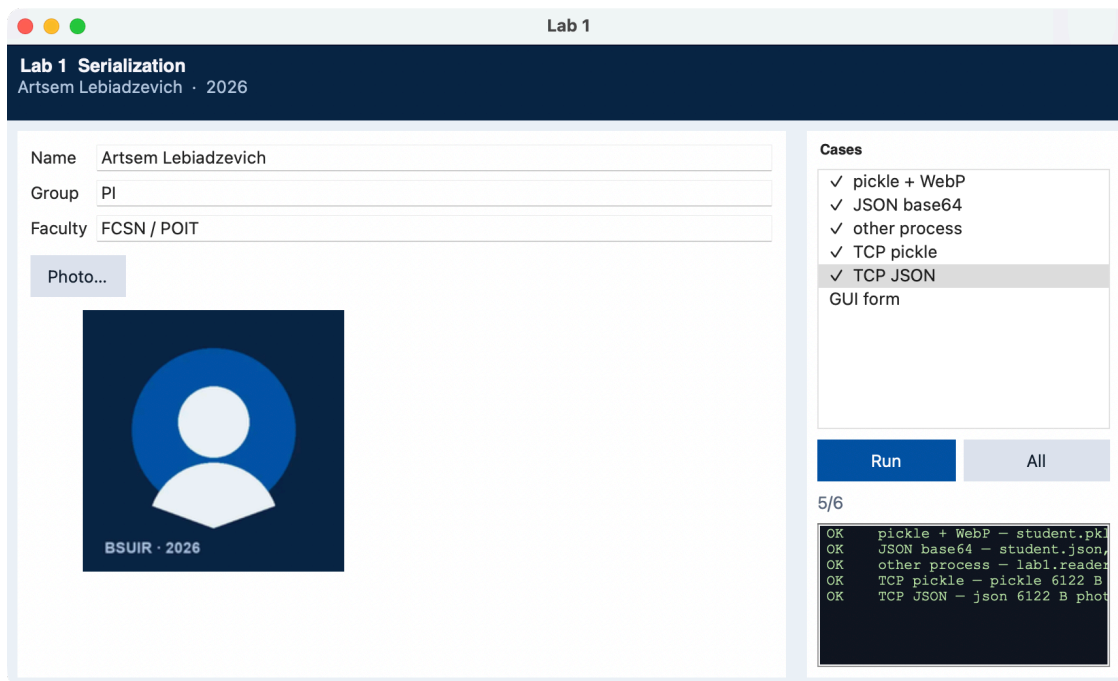


Рисунок 5.1 – Окно лабораторной работы после выполнения сценариев

Команда `python main.py --lab 1` выполняет те же действия без окна (листинг 5.1).

Листинг 5.1 – Вывод программы

```
Student: Artsem Lebiadzevich, PI, FCSN / POIT
Photo: avatar.webp, image/webp, 6122 bytes
pickle file: student.pkl (6320 bytes)
JSON file: student.json (8360 bytes)
Other process (reader) pickle: Artsem Lebiadzevich, PI, FCSN / POIT
JSON roundtrip: Artsem Lebiadzevich, PI, FCSN / POIT, photo=Photo(filename='avatar.webp', mime='image/webp', bytes=6122)
GUI snapshot: {'title': 'Visual form', 'button_label': 'Click me', 'text_value': 'Artsem Lebiadzevich', 'label_text': 'demo', 'extra': {}}
Lab1 server listening on 127.0.0.1:8003
client ('127.0.0.1', 52437)
Network JSON echo: Artsem Lebiadzevich, PI, FCSN / POIT
client ('127.0.0.1', 52438)
Network pickle echo: Artsem Lebiadzevich, PI, FCSN / POIT
Photo sent over the wire: 6122 bytes (port 8003)
Lab 1 done
```

Файл pickle занимает 6320 байт, файл JSON — 8360 байт при размере изображения 6122 байта: разница объясняется кодированием base64. Соответствие пунктам задания приведено в таблице 5.1.

Таблица 5.1 – Выполнение пунктов задания

Пункт	Задание	Где выполнено
1	класс «Студент»	lab1/models.py
2	создание объекта	окно и lab1/runner.py
3	сериализация в двоичный файл	artifacts/student.pkl
4	десериализация в другом приложении	python -m lab1.reader
5	передача по сети	lab1/server.py, lab1/client.py
6	десериализация на принимающей стороне	сервер и клиент восстанавливают объект
7	визуальный объект	GuiFormState, artifacts/gui_form.json
8	pickle и JSON отдельно	сценарии для каждого формата

Автоматические тесты проверяют совпадение байтов изображения после восстановления из pickle и из JSON, передачу сообщения длиной 5000 байт, превышающей 1024 байта, и ответ сервера. Все тесты проходят.

6 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое сериализация и десериализация? Сериализация — преобразование объекта в последовательность байтов для хранения или передачи. Десериализация — восстановление объекта из этой последовательности.

2. Какие платформы используются для сериализации? В Python — модули `pickle`, `json` и `marshal`. В других средах применяются `Serializable` в Java, `System.Text.Json` в .NET, а также независимые от языка форматы XML и Protocol Buffers.

3. В чём польза сериализации? Она позволяет сохранять состояние программы, обмениваться объектами между процессами и машинами, кэшировать результаты и передавать сообщения. На ней основаны удалённые вызовы и компонентные технологии.

4. Можно ли сериализовать объект, у которого есть методы? Да. Формат `pickle` сохраняет данные объекта и имя класса, а методы после восстановления берутся из класса, который должен быть доступен принимающей программе. В JSON попадают только данные; методы появляются, когда из словаря создаётся объект класса.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены сериализация и десериализация объектов и выполнены все пункты задания для форматов pickle и JSON: объект класса «Студент» сохранён в файл, восстановлен отдельным приложением, передан по сети и восстановлен на принимающей стороне, а состояние визуальной формы сохранено и использовано для создания нового окна. Дополнительно вместе с объектом передаётся изображение WebP.

Практические выводы работы следующие. Формат pickle удобен внутри одной программы на Python, но не годится для обмена с другими языками и недоверенными источниками; на границе систем применяется JSON, где двоичные данные требуют base64 и увеличивают объём примерно на треть. При передаче по TCP длину сообщения нужно указывать явно, иначе объект, не поместившийся в один вызов recv, будет получен не полностью. Сериализация является основой следующих тем курса: удалённого вызова и компонентных технологий, где через границу процесса передаётся уже не объект, а вызов его метода.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Герман, О. В. Сериализация и десериализация: лекция и лабораторная работа по дисциплине «Технологии компонентного программирования» / О. В. Герман. — Минск: БГУИР, 2024.
2. Лутц, М. Программирование на Python: в 2 т. / М. Лутц. — 4-е изд. — СПб.: Символ-Плюс, 2011. — Т. 2. — 992 с.
3. Python documentation. pickle — Python object serialization [Электронный ресурс]. — Режим доступа: <https://docs.python.org/3/library/pickle.html>.
4. Python documentation. json — JSON encoder and decoder [Электронный ресурс]. — Режим доступа: <https://docs.python.org/3/library/json.html>.